

SmartHire GenAI — Final Project Report

1. Overview

SmartHire GenAI is a resume-matching and career-mentoring portal built around six modules: a structured resume parser, semantic job search over a FAISS index, an AI-generated CV improvement tool, a RAG-based career mentor, a guardrails safety layer, and a Streamlit web portal tying it all together.

2. Design Choices

Model selection: gemini-2.5-flash-lite was used for all generation tasks (resume parsing, CV suggestions, mentor answers), and models/gemini-embedding001 for all embeddings. These were chosen for free-tier availability during development, balancing capability against rate limits.

Resume parsing: rather than relying on the LLM's raw output, the parser enforces a strict JSON schema via prompt instructions, then validates the result against expected fields before use. An explicit instruction was added to help the model infer a candidate's likely target role from their skills and experience, since this is rarely stated directly on a resume - this measurably improved downstream job-search relevance (documented in the evaluation report).

Job search: job postings are combined (title + skills + description) into a single text block per job, embedded, and indexed in FAISS using L2 distance. Due to free-tier embedding rate limits, the index was built from a partial sample (~300-350 jobs) rather than the full dataset.

CV suggestions: a small prompt library (prompts.py) defines a single structured prompt asking for three specific outputs - missing skills, weak bullet points rewritten with stronger language, and a tailored summary - rather than open-ended advice, to keep output consistent and gradable.

AI Career Mentor (RAG): the mentor retrieves from two sources - a career notes knowledge base (7 industries, 21 sub-roles, written specifically for this project rather than scraped) AND the same FAISS job postings index used by semantic job search.

This means the mentor can answer both general career-guidance questions and questions about specific currently-available jobs, citing which job postings or documents each answer draws from. Generation is orchestrated with LangChain (a ChatPromptTemplate composed with a Gemini chat model via LCEL), rather than calling the Gemini API directly. The mentor's system prompt strictly instructs it to answer only from retrieved context and to say it doesn't know rather than guess.

Guardrails: a two-layer approach was used - fast, free rule-based checks (empty/oversized input, keyword blocklist using word-boundary matching to avoid false positives, prompt-injection detection) followed by one LLM-based topic classification call for subtler off-topic input. A dedicated self-harm detection path was added to route distressed users to real crisis resources instead of a generic refusal - this was identified as a gap during testing and fixed before final submission.

Notebook structure: rather than embedding all index-building logic directly in the notebooks, the FAISS-building code lives in standalone scripts (build_job_index.py, build_notes_index.py) at the project root. The notebooks import and demonstrate this logic on small samples instead of re-running full embedding jobs, avoiding repeated consumption of free-tier API quota while still fulfilling the notebook requirement for exploration and documentation purposes.

3. What Worked Well

- The resume parser reliably produced valid, schema-conformant JSON across many different real resumes and formats, with an automatic retry on malformed output.
- Job search retrieval performed well (76-84% relevance, averaging 60-80% hit rate) across most tested industries (IT, Finance, Sales, HR).
- The RAG mentor correctly grounded its answers in both the career notes and real job postings and, importantly, correctly refused to answer questions where retrieved context existed but didn't actually support an answer (tested explicitly - see evaluation report) - demonstrating genuine grounding rather than naive retrieval-then-generate behavior.
- Guardrails correctly handled violent-intent, off-topic, and self-harm inputs differently and appropriately, including a mid-project fix to give self-harm messages real crisis resources instead of a flat refusal.

4. Limitations

- The job index (~300-350 postings) is a subset of the full dataset due to free-tier API rate limits, not the complete corpus the dataset offers. Testing showed this concretely hurt results for underrepresented fields - a Healthcare/Public Health resume scored 0% relevance in the top 5 matches, while common corporate fields (IT, Finance, Sales) scored 60-80%.
- The career notes knowledge base covers 7 industries; questions outside those are correctly refused rather than answered, which is intentional (avoiding hallucination) but limits the mentor's practical coverage.
- RAG retrieval occasionally misses a relevant chunk when a question's wording differs significantly from the source text's wording, even when the answer exists in the knowledge base (one specific case is documented in the evaluation report, with an explanation and possible fixes).
- The system was built and tested using free-tier API quotas, which constrained both job index size and the pace of testing/evaluation.

5. Possible Future Improvements

- Rebuild the job index with the full dataset (1,000+ postings) given a paid tier or more time, particularly to fix weak coverage in underrepresented industries like healthcare.
- Increase retrieved chunk count (TOP_K_CHUNKS) or use smaller, more focused chunks to reduce the retrieval-miss issue identified in testing.
- Expand the career notes knowledge base to more industries.
- Implement the stretch goals: conversation memory across mentor questions, citing which specific job/document supports each mentor answer, and a "rewrite my resume for this job" feature.